Introduction to Robotics — Homework 1

The AR-601M arm: forward and inverse kinematics, Jacobian, singularities

Oleg Bulichev

This handout is the detailed statement of Homework 1, covering weeks 1–3. The deck (`In2Ro_hw01.tex`) points back to this document; everything you need to actually do the homework is here. Read §3 and §4 before you write any code, and read §5 only after you are stuck on the identity it hints at, not before.

Contents

1 The robot and the problem **1**

2 Model data **2**
2.1 The factor chain 2
2.2 Link lengths — declared nominal 4
2.3 Joint limits 4
2.4 Individualisation 5

3 Deliverables and the packaging contract **5**

4 Tasks 1–6 **6**

5 Hints **8**

6 Submission format **9**

1 The robot and the problem

AR-601M is a Russian-made anthropomorphic robot with 41 degrees of freedom overall (Figure 1). This homework treats one of its two arms in isolation: a 6-DOF serial chain, shoulder to tool, with a spherical wrist. Everything below is about that one 6-DOF chain, not the whole robot.

Two questions define the whole homework, and it is worth stating both before either has an equation attached. *Forward* kinematics asks: given the six joint angles, where is the tool, and how is it oriented? This question always has exactly one answer, and computing it is a straight-line calculation with no branching — Task 2. *Inverse* kinematics asks the opposite question: given where you want the tool, what joint angles put it there? This question can have zero, one, or several answers for the same target, and answering it is most of the weight of this homework — Tasks 3 and 4, by two different routes. Once both directions are in hand, Task 5 asks how a small joint motion maps to a small tool motion (the Jacobian), and Task 6 asks where that map breaks down (a singularity, where some tool motion becomes momentarily unreachable no matter how the joints move).

This arm is a convenient teaching example for exactly one structural reason: its last three joints share a single point in space, the *wrist centre* (§2.1). That one fact is what lets Tasks 3–5

be solved by closed-form algebra and cross-checked by hand, rather than requiring a general-purpose numerical solver from the first step the way an arbitrary 6-DOF chain would. Losing sight of that fact is the most common way to make this homework harder than it is.

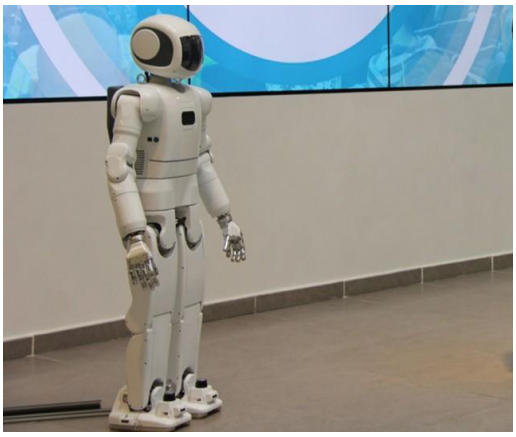


Figure 1: The AR-601M arm.

The arm's geometry comes from the instructor's own hand-annotated photograph of the manufacturer's blueprint (Figure 2). It is an angled phone photograph of a paper drawing, not a clean engineering export: the dimension callouts are cut off by the frame edge, and only a 70° angle mark and fragments of component mass labels survive. Read it for the *joint axes* it marks in blue pen, not for lengths — §2 explains where the lengths in this homework actually come from.

Figure 3 is a second, original drawing: the kinematic scheme read *off* Figure 2, not a redraw of it. It names the six joint axes, the wrist centre, the four link offsets, the fixed shoulder tilt and the left/right mirror — the model Task 1 asks you to reproduce for yourself, in your own hand, before you look at this one again.

2 Model data

2.1 The factor chain

From the blueprint's own equation for the arm (reproduced from `AR-601M arm REPORT.pdf`, its equation (1)):

$$ {}^{0}T_{6} = R_z(t_1)\,T_z(l_1)\,R_z(10^\circ b)\,R_y(t_2)\,T_y(-l_2 b)\,R_x(t_3)\,T_z(-l_3)\,R_z(t_4)\,R_y(t_5)\,R_x(t_6)\,T_x(l_4) \qquad (1) $$

where $b = +1$ for the right arm and $b = -1$ for the left arm. Joints t_4, t_5 and t_6 carry no translation between them, so their three axes meet at one point: the arm has a spherical wrist and the six-joint problem decouples into two three-joint problems that share only that point. This fact is the entire content of Task 3's first step, and it is visible directly in (1): find it there before you read any further.

Notation. $R_z(\theta)$, $R_y(\theta)$ and $R_x(\theta)$ are the standard 4×4 homogeneous rotation matrices about the named axis of the *current* frame; $T_x(d)$, $T_y(d)$ and $T_z(d)$ are homogeneous translations along the named axis, also of the current frame. (1) is read left to right as a sequence of moves starting from the fixed base frame, each one expressed in the frame the previous move left you in — this is why the matrices are *post*-multiplied, not pre-multiplied, and why the order in (1) cannot be permuted. $q = (t_1, \ldots, t_6)$ denotes the six joint values, always in radians internally, however you choose to print them. b is not a joint and is never part of q; it is a fixed property of which physical arm you are modelling, chosen once and held fixed for an entire run of `FK`, `IK` or `Jacobian`.

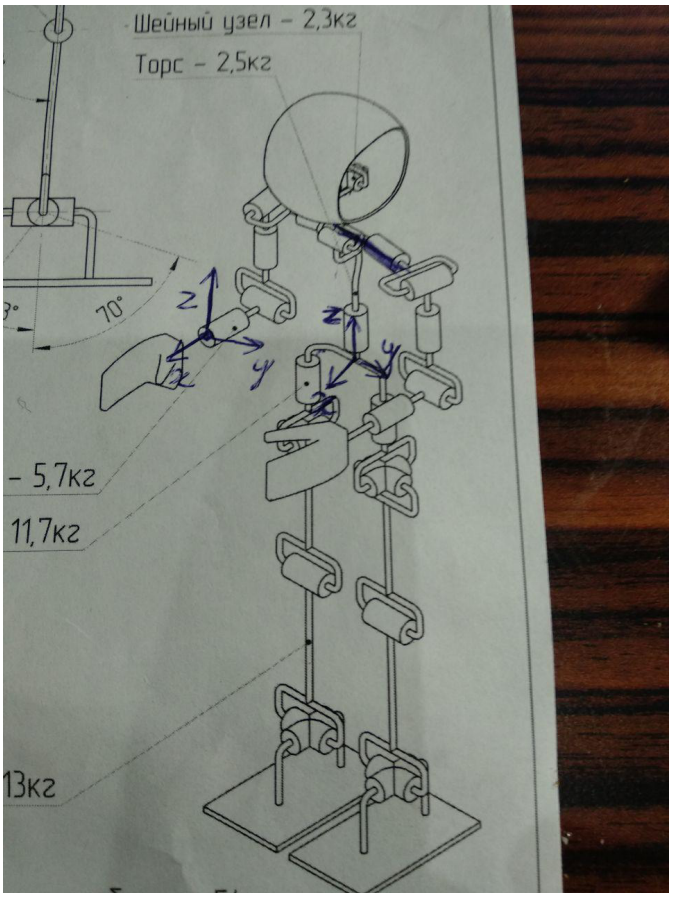


Figure 2: The blueprint, with the instructor's own axis annotations in blue. Dimension callouts are cut off by the frame edge; only 70° and mass-label fragments are legible.

Reading the chain term by term. It is worth walking (1) once, factor by factor, before implementing it, since every factor is either a joint or a fixed geometric fact and confusing the two is the single most common modelling error in Task 1:

Factor	What it is	Fixed, or a joint?
$R_z(t_1)$	base rotation about z	joint t_1
$T_z(l_1)$	base-to-shoulder offset	fixed (l_1)
$R_z(10°b)$	shoulder tilt about z	fixed (10°, sign by b)
$R_y(t_2)$	shoulder rotation about y	joint t_2
$T_y(-l_2 b)$	shoulder offset, sign by b	fixed (l_2)
$R_x(t_3)$	elbow rotation about x	joint t_3
$T_z(-l_3)$	upper-arm offset	fixed (l_3)
$R_z(t_4)$	wrist rotation about z	joint t_4
$R_y(t_5)$	wrist rotation about y	joint t_5
$R_x(t_6)$	wrist rotation about x	joint t_6
$T_x(l_4)$	wrist-to-tool offset	fixed (l_4)

Six factors are joints, five are fixed geometry, and one fixed factor — the 10° tilt — sits in the *middle* of the chain, between t_1 and t_2, rather than at either end. A model file that lists it as a seventh joint, or that folds it into l_1 or l_2 instead of leaving it as its own rotation, will not reproduce (1) even though every length and angle in it is individually correct.

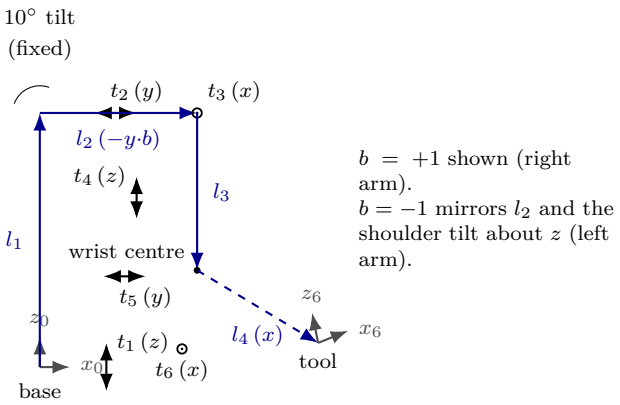


Figure 3: The kinematic scheme, drawn from the factor chain of §2. Not to scale.

2.2 Link lengths — declared nominal

The blueprint photograph does not give legible link lengths (see Figure 2's caption). Rather than invent measurements, this homework **declares** nominal values:

Symbol	Meaning	Nominal value
l_1	base to shoulder, along z	0.40 m
l_2	shoulder offset, along $-y \cdot b$	0.18 m
l_3	upper-arm offset, along $-z$	0.30 m
l_4	wrist centre to tool, along x	0.12 m

and a fixed shoulder tilt of 10° about z, applied between t_1 and t_2 in (1) and **not** itself a joint. $b = +1$ selects the right arm, $b = -1$ the left; Figure 3 shows the right arm, and the left arm is its mirror image across the x-z plane — l_2's offset and the shoulder tilt both flip sign, nothing else does.

These nominal values play two distinct roles, and keeping them apart matters. First, they are the **reference geometry**: Task 1's model file, Task 2's forward-kinematics check poses, and every worked identity in §5 all use exactly $l_1 = 0.40$, $l_2 = 0.18$, $l_3 = 0.30$, $l_4 = 0.12$ m, because that is the one configuration everyone — you, the instructor, the reference solution — can agree on and cross-check against. Second, l_1 and l_4 stay at these nominal values for the *entire* homework, on every task: only l_2 and l_3 change, and only from §2.4 onward, where Tasks 3 and 4 ask you to re-solve the same arm with *your own* l_2, l_3 substituted in place of the nominal pair above. Nothing about the method changes when you make that substitution — (1) does not care what numbers you put into it — but every reported number from Task 3 onward should be computed with your individualised pair, not the nominal one, unless a step explicitly says otherwise (Task 1 and Task 2 do not; they are the shared baseline).

2.3 Joint limits

`AR-601M arm SLIDES.pdf` physical page 3 lists six numbers under "Angle limits": 90°, 105°, 45°, 90°, 90°, 90°, one per joint in order, with no statement of whether each is a half-range or a full range. **This course reads each number as a half-range**: joint t_i is limited to $[-\text{value}_i, +\text{value}_i]$, not $[0, \text{value}_i]$. The reasoning: a servo-driven joint with a one-sided range would put its own zero pose — the pose every $R(\cdot)$ factor of (1) evaluates to the identity at — at one physical extreme of travel rather than at its centre, which is an unusual design choice for a joint that has to reach targets on both sides of straight ahead. This is not proof — the source page genuinely does not say — which is exactly why it is stated here as an adopted course convention rather than left implicit in a model file's numbers; a reader of the original slide is

free to reach a different conclusion, and should say so explicitly if they build a model file around it. The resulting table:

Joint	Axis	Lower limit	Upper limit
t_1	z	$-90°$	$+90°$
t_2	y	$-105°$	$+105°$
t_3	x	$-45°$	$+45°$
t_4	z	$-90°$	$+90°$
t_5	y	$-90°$	$+90°$
t_6	x	$-90°$	$+90°$

2.4 Individualisation

Every student solves the same arm, by the same method, in (1) above. What differs per student is the data: l_2, l_3 and the insertion axis a of Task 3/4's target line are taken from the row matching your position in the class list, distributed with the assignment. The table below is printed with its column headings only — the instructor fills in the rows when the assignment goes out; nothing below is a placeholder value to use by mistake.

Position in class list	l_2 (m)	l_3 (m)	Insertion axis a

Table 1: Per-student parameters. Distributed separately; left empty here on purpose.

Before using your assigned row, check it, the same way you would check any input you did not compute yourself: your l_2 and l_3 should both be positive lengths in metres of roughly the same order of magnitude as the nominal pair in §2.2, and a should already be a unit vector (if it is not, normalise it and note that you did). None of these checks require solving anything — they catch a transcription slip in the distributed table, not an error in your own work, and they take under a minute against the minutes an undetected typo would otherwise cost you partway through Task 3.

3 Deliverables and the packaging contract

What you submit is graded on its *packaging*, not on the language you wrote it in:

1. A `pixi.toml` **or** a `Dockerfile` at the repository root.
2. **One command** reproduces every number your report quotes, from a clean checkout: `pixi run check`, or a documented `docker build` followed by `docker run`.
3. Three functions exist, with exactly these roles: `FK(q, robot)`, `IK(T, robot)`, and `Jacobian(q, robot)` — where `robot` is read from a serial-chain model file of your own (URDF-like, but yours).

The homework's own `input_data/` folder ships a worked Python example of this contract: a `pixi.toml` and a `Dockerfile` side by side, an `ar601m.yaml` model file in the shape Task 1 asks you to build your own version of, three stub modules carrying the `FK`/`IK`/`Jacobian` signatures as `NotImplementedError`s, and a contract test. Read that contract test, not the stubs: it checks only the *shape* of a correct submission — that the model loads with the right joint count and order, that each function accepts the stated arguments and is callable — and never the numeric

answer. It is an example, not a mandate: submit C++, Julia, MATLAB or anything else, as long as your own repository satisfies the same three points above.

Why packaging rather than language is what is graded: this homework's numbers only mean something if the person reading your report can reproduce them, and "read the source, trust the arithmetic" does not scale to an entire class. A single command from a clean checkout is the one thing that lets grading, and later your own future self checking a result six months on, skip re-deriving your environment by hand. A correct derivation whose numbers cannot be reproduced this way is graded as if the numbers were never shown — see §6's checklist for exactly how that plays out.

4 Tasks 1–6

Each task below states what you hand in, what is required of it, and the one-line criterion the instructor checks first. About 18 hours of work across the three weeks this homework spans.

The six tasks are not six independent exercises glued together; they build in a fixed order and each one hands the next its own working result. Task 1 gives every later task its model file and (1) in code form. Task 2's `FK` is what Tasks 3 and 4 check every returned pose against — an `IK` that has never been round-tripped through your own `FK` is unverified, not merely unpolished. Task 3's analytical branches are what Task 4's numerical solver is compared against, in both directions: they confirm the numerical answers, and the numerical answers confirm nothing was missed among the analytical ones. Task 5's Jacobian is evaluated at configurations Task 6 later sweeps across, and Task 6's map is only informative once Task 5's three independent routes already agree with each other. A task report that skips straight to its own numbers, without saying which earlier task's output it consumed and how it was checked, is missing the one thing that makes this homework's numbers trustworthy rather than merely plausible.

Report each task in the same shape: state your assumptions before your result (which pose, which arm, which student row), show the intermediate quantity the "done when" line asks for — not only the final answer — and say explicitly how you checked it, even when the check is only a round trip through your own `FK`.

Task 1. Kinematic model. *Statement:* from Figure 1 and Figure 2, name the six joints and their axes, write (1) out by hand before touching any code, and build your own robot-model file carrying the six joints, their limits (§2.3) and the four link lengths (§2.2). *Requirements:* the factor chain is written symbolically first; the model file's joint order matches it; every length is stated as declared nominal, not measured. *Done when:* your model file loads and lists six revolute joints and four links, each with the right axis and limit. *Pitfall:* the fixed 10° shoulder tilt is not a joint and must not appear in your model's joint list — it belongs inside `FK` as a constant factor of (1), between t_1 and t_2, exactly once.

Task 2. Forward kinematics. *Statement:* implement `FK(q, robot) -> T` from Task 1's chain. Evaluate it at three configurations — the zero pose, one joint at its declared limit, and a generic pose of your choosing — and report $R^\top R = I$ and $\det R = 1$ for each, plus the generic pose's orientation as ZYX angles. Add a visualizer that accepts six joint angles (radians) and shows the correctly connected arm, its base and tool frames, and the resulting homogeneous transform T. *Requirements:* the three configurations are fixed for the rest of the homework, since Task 3 checks its answers against them. State the ZYX convention explicitly: $R = R_z(\psi)R_y(\theta)R_x(\phi)$ for yaw, pitch, roll. Report all three visualizer examples with images and their numerical transforms. *Done when:* all three orthonormality residuals are reported numerically (not asserted) within your stated tolerance, and a reader can run the visualizer on each saved angle set. *Pitfall:* a residual that is small but not machine-precision small

(say, 10^{-6} instead of 10^{-15}) usually means an angle was carried in degrees somewhere it should have been radians, not that floating point is merely "a bit noisy" — track it down rather than widen your tolerance to hide it.

Task 3. Analytical inverse kinematics. *Statement:* derive, by hand, the wrist centre from a target pose, the shoulder sphere that depends on t_3 alone, the closed-form solution for t_1 and t_2 given t_3, the line-sphere step for an insertion-axis target, the full branch count, and the two degeneracies a robust implementation must handle explicitly. Implement `IK(T, robot) -> list[q]` returning every unique, valid isolated configuration. Accept either a pose produced by Task 2's `FK`, or position (x, y, z) in metres plus ZYX yaw ψ, pitch θ, roll ϕ in radians, with $R = R_z(\psi)R_y(\theta)R_x(\phi)$. *Requirements:* exercise both arms ($b = +1$ and $b = -1$), derive the closed-form t_1, t_2 step yourself, apply the declared joint limits, normalize equivalent angles, and deduplicate coincident configurations. Draw every valid branch, labeling each picture with all six joint angles and its translation and rotation residual from `FK`. Distinct branches that overlap in the picture still need separate labels. State your numeric tolerances; defaults are 10^{-6} m translation, 10^{-6} rotation Frobenius norm, and 10^{-6} rad circular angle deduplication. A non-isolated singular family is marked separately, rather than sampled and counted as finite. *Done when:* the generic target has a complete table and pictures of all unique limit-valid branches, with each round-trip residual within the declared tolerance. A numerical solver's failure is not proof that a target is unreachable. *Pitfall:* unconstrained algebraic branches can merge or be removed by limits, so the feasible count need not equal the algebraic maximum.

Task 4. Numerical inverse kinematics. *Statement:* implement a numerical solver (Newton with the pseudoinverse, or damped least squares) from at least two different seeds, and compare its converged answers against Task 3's analytical branches for the same target. *Requirements:* the two seeds land on two different branches; a run that fails to converge is reported as a failure, not dropped silently. *Done when:* both runs' final residual, iteration count and nearest analytical branch are reported. *Pitfall:* choosing both seeds close together, or both close to the same analytical branch, is a common way to accidentally satisfy the letter of "two seeds" while never actually testing that the numerical method can find a different branch than the one you happened to start near.

Task 5. Jacobian. *Statement:* implement `Jacobian(q, robot) -> J`, the 6×6 geometric Jacobian, by two independent routes, and cross-check both against a finite-difference approximation of `FK`. *Requirements:* the two routes are genuinely independent implementations; the cross-check includes a configuration near a singularity (Task 6). *Done when:* the three routes' pairwise residuals are reported at a stated configuration, with the finite-difference step size stated. *Pitfall:* "two independent routes" means two genuinely different derivations of the same 6×6 matrix (for instance, the geometric axis-and-offset construction against a direct differentiation of `FK`'s own matrix product) — not the same derivation typed out twice, and not one route obtained by copying the other's code with the variable names changed.

Task 6. Singularity and solution-count maps. *Statement:* show two labeled panels: a joint-space sweep of $1/\mathrm{cond}(J)$, and a Cartesian position slice counting unique valid IK configurations at a fixed, stated tool orientation. Separately, draw a stick figure of one configuration showing six joint axes and link offsets. *Requirements:* state the arm, joint limits, slice coordinates, held angles or orientation, grid step, and numeric precision. The count panel uses a continuous palette with a numeric legend from zero

to the maximum finite count attained in this case; no particular colors or fixed count of eight are required. Mark a proven unreachable target as zero, an unresolved numerical failure as unknown, and any non-isolated singular family as ∞ or "continuum", outside the finite palette. A count of zero requires an exhaustive analytical check, not failed numerical convergence. At one target, different IK branches can have different Jacobian conditioning. State in the report that the first panel is ambiguous in Cartesian space because different configurations can reach the same position. Euler-angle gimbal lock alone does not establish a manipulator Jacobian singularity. *Done when:* both reproducible panels, legend, stick figure, slice/limit/tolerance settings and the ambiguity sentence are present.

5 Hints

Use these only once you are stuck, not as a first move. One identity of the analytical route — the closed-form solve for t_1 and t_2 once t_3 is known — is deliberately *not* hinted here: deriving it is Task 3's own work.

The four hints below are ordered the way the solve actually proceeds, not the way they are numbered against the design document's own internal identities (there is a Hint 1, 2, 4 and 5 here and deliberately no Hint 3 — see above). Read them in order: Hint 1 collapses a 6-DOF unknown pose down to a 3-DOF point in space; Hint 2 collapses that point down to a single scalar equation in t_3 alone; only once t_3 is a number does the un-hinted step for t_1, t_2 become a two-unknown, two-equation problem instead of a three-unknown one. Hint 4 addresses a separate concern: the insertion-axis variant of the problem poses the target as a line, not a single point, and Hint 4 is what turns that line into the same shoulder-sphere equation Hint 2 already set up — it is used together with, not instead of, Hints 1–2. Whichever step you are stuck on, the single most useful check available to you is not re-deriving it by a different route on paper, but taking whatever $(t_1, \ldots, t_6)$ you believe you have found and running it through your own `FK`: if the resulting pose does not match the target you started from to near machine precision, the error is somewhere in the chain of hints above, not in `FK` (Task 2 already checked `FK` on its own, independently, before Task 3 began).

Hint 1 — recovering the wrist centre

The tool offset $T_x(l_4)$ is applied along the tool frame's own x-axis, which is the first column of the target rotation R. With ${}^0T_6 = \left(\begin{smallmatrix} R & p \\ 0 & 1 \end{smallmatrix}\right)$, back that offset out of the target position directly:

$$W = p - l_4 \, R_{:,0}$$

No trigonometry, no iteration — one vector subtraction.

Hint 2 — the shoulder sphere

Let $S = (0, 0, l_1)$ be the shoulder point and $V = W - S$. Neither t_1 nor t_2 changes $\|V\|$ — they only reorient V, since both are rotations about an axis through S. What sets the length is the sub-chain $T_y(-l_2 b)\, R_x(t_3)\, T_z(-l_3)$, i.e. t_3 alone:

$$\|W - S\|^2 = l_2^2 + l_3^2 - 2\, b\, l_2\, l_3 \sin(t_3)$$

Solve this for t_3 before attempting t_1 or t_2.

Hint 4 — the line-sphere step

When the target is posed as an insertion along a line $q_0 + t\,a$ (unit a, free depth t), the wrist centre must lie on that line *and* on the shoulder sphere of Hint 2 (radius r, centre S). Substituting the line into $\|P - S\|^2 = r^2$ gives a quadratic in t:

$$t^2 + 2t\left(a \cdot (q_0 - S)\right) + \|q_0 - S\|^2 - r^2 = 0$$

with 0, 1 or 2 real roots. The reachable radii form a spherical shell $|l_2 - l_3| \le r \le l_2 + l_3$ — derive this bound yourself and check it against your own l_2, l_3.

Hint 5 — how many branches

The arm and wrist can each have several regular branches. The residual ZYX wrist rotation has two regular decompositions: derive both from $R_z(t_4)R_y(t_5)R_x(t_6)$ and verify each by multiplication. Count unique configurations only after applying the declared limits and checking the complete pose with `FK`; merged branches count once. At a singular family the answer may be non-isolated.

6 Submission format

Submit a link to your repository (packaged per §3) containing:

- `pixi.toml` or `Dockerfile` at the root;
- your `src/` (or equivalent) implementing `FK`, `IK`, `Jacobian`, both maps and the Task 2–3 visualizer;
- your tests, reproducible by the one command of §3;
- a report (PDF or Markdown) covering Tasks 1–6 in order, with derivations, reproduced numbers, Task 2's three angle-to-arm-and-T examples, Task 3's branch pictures and angle/residual table, and Task 6's two panels, legend, settings and ambiguity sentence.

There is no minimum page count for the report, and no credit for reaching one: a two-page report that states every "done when" criterion's evidence plainly outscores a fifteen-page report padded with restated theory that never states whether its own numbers satisfy the criterion. Write for a reader checking your work against §4, not for a reader being persuaded of effort.

Before you submit, it is worth spending five minutes on a short, mechanical check, since every item below has been the actual cause of a lost grade in earlier runs of this course: does `pixi run check` (or your own equivalent single command) succeed from a truly clean checkout, not just in the directory you have been working in all along; does every number your report quotes actually appear in that command's output, rather than in a scratch notebook you forgot to fold back in; does your report state which arm ($b = \pm 1$) and which row of §2.4's table each result used; can the one command regenerate the Task 2–3 examples and both Task 6 panels; and does the report carry the Task 6 ambiguity sentence? None of this is new content — it is every requirement already stated in §4, collected once here as a final pass.

Submit by the deadline announced on the course page.